

HERALD: Hazard Estimation via Real-time Analysis of Logit Distributions

Joaquin Campo¹ Ernesto Dufrechou¹ Guillermo Moncecchi¹

¹Instituto de Computación, Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay

{joaquin.campo, edufrechou, gmonce}@fing.edu.uy

<https://github.com/joaquinCampo/HERALD>

<https://huggingface.co/datasets/Jocana/herald-logits>

Abstract

Heavy KV-cache compression makes large language models fail catastrophically: they get stuck in loops or fail to terminate. We show these failures are highly predictable from the model’s own logits, dozens of tokens before they become visible. We introduce HERALD, a lightweight XGBoost classifier trained on zero-cost per-token logit features. Across six KV-cache compression methods applied to Qwen2.5-7B-Instruct on GSM8K, it forecasts catastrophic failures with AUROC 0.97, generalizes to held-out compressors via leave-one-compressor-out evaluation, and retains AUROC 0.88 in the pre-onset regime, when no failure is yet visible, compared to a rolling-entropy baseline that barely beats chance. These results show that catastrophic compression failures emit a clear warning signal well before they become observable, opening a path to runtime intervention with negligible compute overhead.

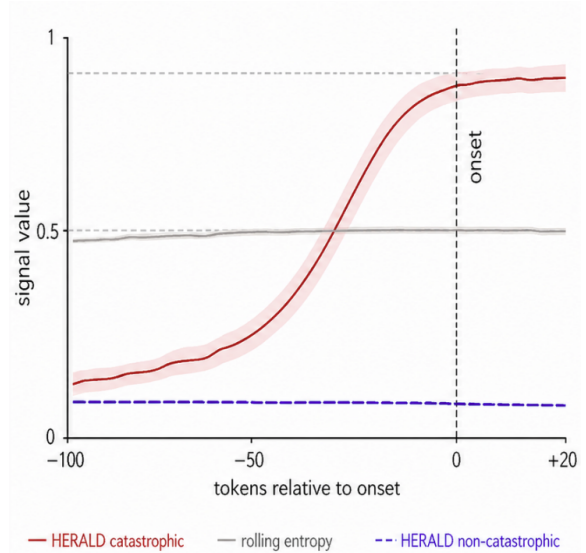
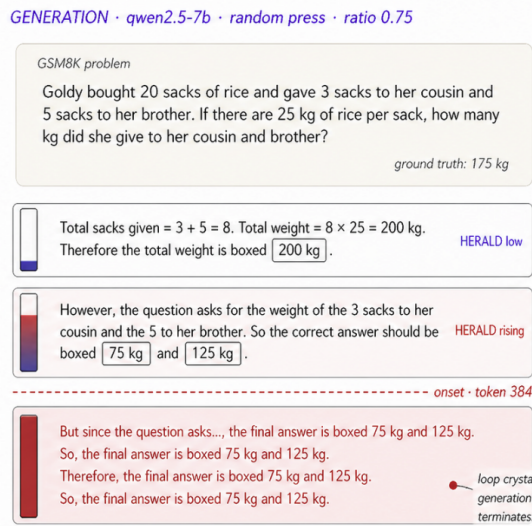


Figure 1: Compression-induced catastrophes are predictable from logits. *Left:* a real GSM8K generation under heavy KV-cache compression (random press, ratio 0.75). The model first answers correctly, drifts into spurious self-correction, and finally collapses into a non-terminating loop. HERALD’s hazard score, drawn as the vertical bar on the left, rises tens of tokens before any failure is visible in the output. *Right:* averaged across all catastrophic sequences in the test set, HERALD diverges from a rolling-entropy baseline well before the onset ($t = 0$); on non-catastrophic sequences, it stays flat throughout.

1 Introduction

Large language models broadcast their impending failure through their own logits. Under heavy KV-cache compression, generations collapse into looping or run past the stop token without ever producing one; long before any of this becomes visible in the output, the next-token distribution begins to deform in characteristic ways. This paper turns that observation into a runnable predictor.

KV-cache compression has become a load-bearing optimization for long-context LLM inference, since memory and decode latency both scale linearly in cache size. A growing family of compressors (StreamingLLM, SnapKV, ExpectedAttention, KNorm, TOVA, and others) trade cache fidelity for throughput, and at moderate ratios they preserve task accuracy well. At heavy ratios the picture changes. Errors stop being graceful: the model loops on a phrase or fails to terminate. These failures are user-facing, hard to roll back, and concentrated in the tails of the metric distributions that the literature tends to report.

Existing tools do not catch this in time. Benchmarks for KV-cache compression report task-level accuracy averaged across prompts, which smooths over catastrophic generations and obscures their structure. Output-level quality monitors and post-hoc detectors only fire once the model has already produced bad text. To our knowledge, no prior work asks whether the next-token distribution reveals a compression-induced catastrophe before it manifests, even though the distribution is computed at every step and is therefore free.

We treat the problem as token-level hazard prediction. At each decoding step we extract a small set of zero-cost features from the logit vector (entropy, top- k probability mass, rank-based statistics, and related quantities) and label the step by whether a catastrophe begins within the next H tokens. We then train HERALD, a lightweight XGBoost classifier, on this stream. We evaluate on a large GSM8K sweep across six KV-cache compressors applied to Qwen2.5-7B-Instruct, with a wide compression-ratio range and a held-out set of compressors used only at test time.

HERALD forecasts catastrophic failures with AUROC 0.97 at the one-token horizon, generalizes to held-out compressors via leave-one-compressor-out evaluation, and retains AUROC 0.88 in the pre-onset regime, when no failure is yet visible, while a rolling-entropy baseline barely beats chance. The signal is therefore not an artifact of any single compressor and not a delayed reflection of bad output already on the page; it is a structural precursor that the model itself produces.

This matters for two reasons. Scientifically, the fact that a single cheap predictor transfers across compressors suggests that compression-induced catastrophes share a latent failure trajectory in logit space, rather than being method-specific quirks. Practically, the predictor runs inside the decode loop with negligible compute overhead, opening a path to runtime mitigation: early stopping, switching decoding strategy, or falling back to a larger cache budget at the moment the warning fires.

Contributions.

1. We characterize the catastrophic failure modes induced by heavy KV-cache compression (looping and non-termination) and operationalize them as token-level hazard labels.
2. We release a large per-token logit dataset spanning six KV-cache compressors and a wide compression-ratio sweep on GSM8K with Qwen2.5-7B-Instruct.
3. We introduce HERALD, a lightweight predictor that forecasts these catastrophes from zero-cost logit features.
4. We show the signal generalizes to unseen compressors under leave-one-compressor-out evaluation and remains informative in the pre-onset regime, where no failure is yet visible.
5. We position HERALD as a runtime intervention substrate: a hazard score that any downstream system can act on while generation is still in flight.

2 Catastrophic failures under KV-cache compression

2.1 What we mean by *catastrophic*

We distinguish two regimes of LLM failure under KV-cache compression. The first is graceful accuracy degradation: the model finishes generating, returns a parseable answer, and is simply wrong. This regime is captured by standard task-accuracy benchmarks and tracked by every published evaluation of KV-cache compression. The second regime is qualitatively different. The generation itself goes wrong: the model emits the same fragment over and over, runs to the token budget without ever producing a stop token, or both. We call this regime *catastrophic*. It is not a measure of how good the answer is; it is a measure of whether the model produced a usable answer at all.

We formalize two catastrophic failure modes:

- **Looping:** the model emits a chunk of tokens, then re-emits the same chunk, then re-emits it again, with no new content in between.
- **Non-termination:** the model never produces an end-of-sequence token within the generation budget.

We do not include three plausible candidates: *wrong-answer* (a normal mode of LLM failure even on uncompressed inference, see Sec. 2.3), *instruction amnesia* (the model solves a different problem than asked), and *coherence collapse* (the output becomes ungrammatical mid-stream). Each is real and occasionally observed in our data, but none admits a robust automatic detector at our scale, and we leave them as future work.

Looping and non-termination earn their place because they are (i) detectable from the token stream alone, with no judge model required; (ii) demonstrably caused by compression, since their rates rise sharply with compression aggressiveness and are near zero on uncompressed runs (Sec. 2.3); and (iii) operationally severe, since both produce unusable outputs and both run the model to its full token budget.

2.2 Detection rules

Non-termination. We mark a generation as non-terminating if the decoding loop exited because it hit the token budget rather than because the model produced an end-of-sequence token, i.e. its stop reason is one of `max_tokens` or `timeout`. Non-termination is a property of the entire trace, not of a moment within it. The detector therefore returns the last emitted token as a placeholder onset, but the hazard-labeling pipeline introduced later shifts this onset earlier in the trace using a fraction parameter (`nt_onset_frac` = 0.75 in our experiments), since predicting the literal final token offers no operationally useful lead time.

Looping. We mark a generation as looping if any window of $W = 20$ contiguous tokens appears at least $r = 3$ times within the generated sequence. The onset of the loop is the start position of the second occurrence of the first window that hits the threshold, i.e. the first token at which the model committed to repeating itself instead of producing new content. The thresholds are calibrated to be tight enough to catch short phrase-level loops (the typical KV-cache catastrophe arc, see Fig. 1, left) and loose enough to ignore natural repetition such as enumerated lists, where short windows can recur without indicating failure. Setting $r = 3$ rather than $r = 2$ is what makes the rule robust to coincidental pairs.

Co-occurrence. The two modes are not independent. A looping generation almost always escalates into non-termination, since once the model commits to a loop it has no mechanism to break out and exhausts the token budget. We report the two detectors separately because they have different temporal signatures (looping has a clear within-trace onset; non-termination does not), but for hazard prediction we treat the union of either firing as a single positive label.

Wrong-answer carve-out. We measure final-answer correctness alongside, but we do not treat wrong answers as a hazard label. The reason is empirical: even with no compression at all, Qwen2.5-7B-Instruct produces a 22.4% wrong-answer rate on GSM8K (112/500 sequences in the uncompressed

baseline, with task accuracy 77.6%). Including wrong answers in the hazard target would inject a base rate of label noise unrelated to compression and would force HERALD to predict a behavior the model exhibits even when nothing is wrong with its cache. We track wrong-answer rate as a separate accuracy metric and otherwise keep it out of the catastrophic label.

2.3 Prevalence across compressors

[Figure placeholder: bar chart of catastrophic rate per compressor, ordered worst to best, with the uncompressed baseline as a horizontal reference line.]

Figure 2: [To be drawn from models/analysis/per_press.json.]

Table 1 reports the fraction of sequences exhibiting at least one catastrophe (looping or non-termination) under each compressor in our sweep. Numbers are sequence-level: a sequence counts as catastrophic if either detector fires anywhere in its trace.

Compressor	Catastrophic rate	$n_{\text{cat}}/n_{\text{seq}}$
KNorm	71.0%	1774/2500
SnapKV	68.7%	1690/2460
Random	35.9%	898/2500
TOVA	15.4%	385/2500
StreamingLLM	13.7%	342/2500
ExpectedAttention	4.8%	121/2500
<i>None (uncompressed)</i>	1.2%	6/500

Table 1: Sequence-level catastrophic rate per compressor on GSM8K with Qwen2.5-7B-Instruct, pooled across all evaluated compression ratios. The uncompressed baseline establishes the model’s intrinsic catastrophe rate on this task.

Three observations stand out. First, catastrophic behavior is highly compressor-specific: there is a roughly 15 $\times$ spread between the most fragile compressor (KNorm at 71%) and the most robust (ExpectedAttention at 4.8%) on the same prompts and the same set of compression ratios. Second, two of the four “principled” compressors in our sweep (KNorm and SnapKV) are *worse than random eviction* in this regime, and not by a small margin: by roughly a factor of two. Random eviction (35.9%) sits below both, which is striking given that random is meant as a sanity baseline rather than a competitive method. Third, the uncompressed baseline is non-zero (1.2%): a small fraction of long-form chain-of-thought generations on GSM8K loop or fail to terminate even with the full cache available. Any predictor of compression-induced catastrophes therefore has to clear an intrinsic floor; the true effect we want to model is the gap between this floor and the per-compressor rates above it.

2.4 Prevalence across compression ratios

[Figure placeholder: line plot of catastrophic rate vs. compression ratio, one curve per compressor, with the uncompressed baseline as a horizontal reference line.]

Figure 3: [To be drawn from models/analysis/per_ratio.json.]

Pooling all compressors at each ratio gives a clean view of how catastrophic behavior scales with compression aggressiveness (Table 2).

Compression ratio	Catastrophic rate
0.000 (uncompressed)	1.2%
0.250	10.1%
0.500	25.5%
0.625	35.9%
0.750	45.9%
0.875	56.7%

Table 2: Sequence-level catastrophic rate vs. compression ratio, pooled across all six compressors evaluated at each ratio. Each 0.125 step in compression adds roughly ten percentage points of catastrophic behavior.

The relationship is approximately linear: each 0.125 step in compression adds roughly ten percentage points of catastrophic behavior, all the way from the floor at zero compression up to 56.7% at the heaviest ratio we evaluate. There is no obvious “safe” heavy regime. By ratio 0.875, more than half of all generations across all six compressors fail catastrophically, which inverts the usual mental model in which catastrophes are tail events to be ignored when reporting average accuracy: at heavy ratios they are the modal outcome.

The pooled view also hides a useful asymmetry that Table 1 surfaces. The headline rate at 0.875 is dominated by the fragile compressors (KNorm and SnapKV); the robust ones (ExpectedAttention) stay tolerable even there. This is precisely the tension that motivates a runtime predictor: an inference system that wants the memory savings of heavy compression cannot trust an averaged per-compressor accuracy number, because the variance across compressors and across individual sequences is enormous and concentrates exactly where it hurts.

2.5 What a catastrophe looks like

Fig. 1 (left) tracks a single catastrophic generation under random eviction at ratio 0.75. The model is asked a one-step GSM8K word problem, solves it correctly in the first part of the trace (200 kg total), then drifts into a spurious self-correction (“However, the question asks for. . .”), and finally collapses into four near-identical repetitions of the same boxed answer before being cut off at the token budget. The arc of correct math, then drift, then loop crystallization, then non-termination is the canonical compression-induced catastrophe shape; both detectors fire on this single trace. The hazard score that HERALD produces, drawn as the vertical bar in the figure, rises tens of tokens before any of the externally visible failure becomes apparent.